



Flight Computer Logic

IndiFlight Flight Computer Mark 1

The core architecture for a reliable amateur flight computer follows a state machine with strict sensor validation gates. Here's the complete logic:

Boot & self-test phase: On power-up, the FC reads the flash chip for any previous session data and dumps it over serial in a formatted log. It then performs a sensor health check — BMP180 reads sea-level pressure baseline, MPU6050 checks for plausible orientation and non-zero data. If sensors fail, buzzer alerts in a distinct pattern and an LED goes solid red. Only a clean sensor handshake advances to idle.

Idle / pre-launch (pad standby): The FC samples BMP altitude and MPU acceleration at 10 Hz, continuously averaging to refine the ground reference. It checks for continuity through the pyro MOSFET line using a low-current sense pin — buzzes and blinks green if continuity is good, red if open. It logs nothing to flash yet (saves write cycles). One crucial gate: it will NOT arm unless continuity is confirmed AND sensors are healthy. The buzzer gives a "ready" chirp pattern. This is your go/no-go signal.

Launch detect (boost phase trigger): The FC polls MPU accelerometer for sustained acceleration above a threshold — I recommend 2.5g for at least 80–100 ms (2–3 consecutive samples at 25 Hz). This prevents engine ignition transients or handling bumps from triggering. On confirmed launch detect, it opens a new flash session with a timestamp/counter, and begins logging at 25–50 Hz: raw altitude (BMP), raw altitude integrated from MPU accel, roll/pitch/yaw (MPU gyro integrated), 3-axis accel, 3-axis velocity (integrated), temperature, pressure.

Coast phase & apogee detection: This is the most critical logic. Apogee is detected by a consensus of two independent methods — never just one:

- BMP method: altitude drops by more than 2–3 meters below the running peak altitude held for at least 200 ms
- MPU method: vertical velocity (integrated accel) crosses zero from positive to negative and stays negative for 200 ms

Both must agree before firing. This dual-vote prevents a pressure transient or vibration spike from causing a premature deployment. The FC maintains a "max altitude" variable updated every sample during coast — never fire if altitude is still climbing.

Apogee fire event: Once both sensors agree on apogee, the MOSFET gate fires for 800 ms (enough to ignite any standard e-match, not long enough to burn the line). A fire event is logged with timestamp and altitude. The buzzer fires a distinct triple-beep

pattern. If continuity is lost after the fire event, it confirms igniter fired — logged as "DEPLOY CONFIRMED." If continuity persists (igniter didn't fire), a re-fire attempt is made once after 1 second, then the FC enters a "DEPLOY FAIL" state and logs it — you need to know this on recovery.

Descent & recovery phase: Post-apogee, logging continues at 10 Hz to conserve flash. The FC watches for landing detection: altitude change less than 0.5 m over 10 seconds AND acceleration near 1g (static). On landing confirmed, logging stops, flash is committed/finalized, and the buzzer begins a periodic locator beep (3 beeps every 10 seconds) to help you find the rocket. This continues until power-off or reset.

Key reliability decisions beyond the hardware: Use a dedicated "arm" check on each boot that requires the pyro continuity sense pin to read open-circuit before any MOSFET drive is possible during testing — so bench testing can never accidentally fire. Add a simple watchdog timer reset in software so a firmware hang doesn't leave the MOSFET gate held high. Store session data with a CRC byte so corrupted flash pages are flagged on readout.

Implementation notes by phase

Boot → Serial dump On setup(), initialize serial at 115200, then scan the flash chip for any written sessions. Each session has a header byte sequence (e.g. 0xAA 0xBB) and a CRC tail. Walk through flash pages, print each session as CSV over serial, then print a separator line like --- NEW SESSION --- before the new one begins.

Self-test gate Before anything else runs: read BMP180, if it returns a pressure value outside 300–1100 hPa, fail. Read MPU6050 — if the who_am_i register returns wrong, fail. Sense pyro continuity (pull-up on a digital pin through a ~10kΩ resistor to the pyro line sense point — reads LOW when continuity exists, HIGH when open). If open circuit on boot, warn but don't hard-fail — let the user know before arming.

Ground baseline averaging In pad idle, run a 30-sample rolling average of BMP altitude over ~5 seconds. This is your groundAlt. Every altitude measurement during flight is rawAlt - groundAlt. This is critical — BMP180 drifts with temperature and the pad may not be at exactly sea level.

Launch detect (critical for not false-triggering)

```

if (accelMagnitude > 2.5g) launchCounter++;
else launchCounter = 0;
if (launchCounter >= 3) → LAUNCH CONFIRMED // 3 samples at 25Hz = 120ms
Apogee dual-vote (most important logic)
// BMP vote
if (currentAlt < (peakAlt - 2.0)) bmpApogeeTimer += loopDt;
else bmpApogeeTimer = 0;
bmpVote = (bmpApogeeTimer >= 200);

// MPU vote
vertVelocity += (vertAccel * loopDt); // integrate accel
if (vertVelocity < 0) mpuApogeeTimer += loopDt;
else mpuApogeeTimer = 0;
mpuVote = (mpuApogeeTimer >= 200);

if (bmpVote && mpuVote) → FIRE
MOSFET fire
digitalWrite(MOSFET_PIN, HIGH);
delay(800);
digitalWrite(MOSFET_PIN, LOW);
delay(200);
if (digitalRead(CONTINUITY_PIN) == HIGH) → DEPLOY_CONFIRMED
else → REFIRE_ATTEMPT

```

Watchdog timer Enable the AVR hardware watchdog at 2 seconds: `wdt_enable(WDTO_2S)` — put `wdt_reset()` in the main loop. This prevents a firmware hang from holding the gate high.

Flash session structure

[0xAA][0xBB][session_id 1 byte][timestamp_ms 4 bytes]

[...data rows: timestamp,bmpAlt,mpuAlt,ax,ay,az,vx,vy,vz,roll,pitch,yaw,temp,pres...]

[EVENT: LAUNCH/APOGEE/DEPLOY/LAND with timestamp]

[CRC 2 bytes][0xCC 0xDD] ← session end marker

Key numbers to tune per your rocket

- Launch threshold: 2.5g (lower for slow-off-the-pad H/I motors, fine as-is for most)
- Burnout detect: 0.5g (below motor's coasting drag deceleration)
- Apogee BMP drop: 2.0 m (can tighten to 1.5m for very high-altitude rockets)
- Apogee vote time: 200ms each
- Fire pulse: 800ms (enough for any standard Estes/Quest/commercial e-match)
- Landing detect: $\Delta alt < 0.5m$ over 10 seconds + accel 0.85–1.15g band

One thing to add to your hardware: a 100 μ F capacitor across the MOSFET gate supply rail. The fire pulse draws a spike of current that can brown-out the Arduino mid-flight. This is a common cause of mystery failures on DIY FCs and a very cheap fix.

Logic Flowchart ahead...

Logic FlowChart

Diagram 1 — Boot through pad idle: power-on, flash readout, self-test gates, and the armed standby loop.

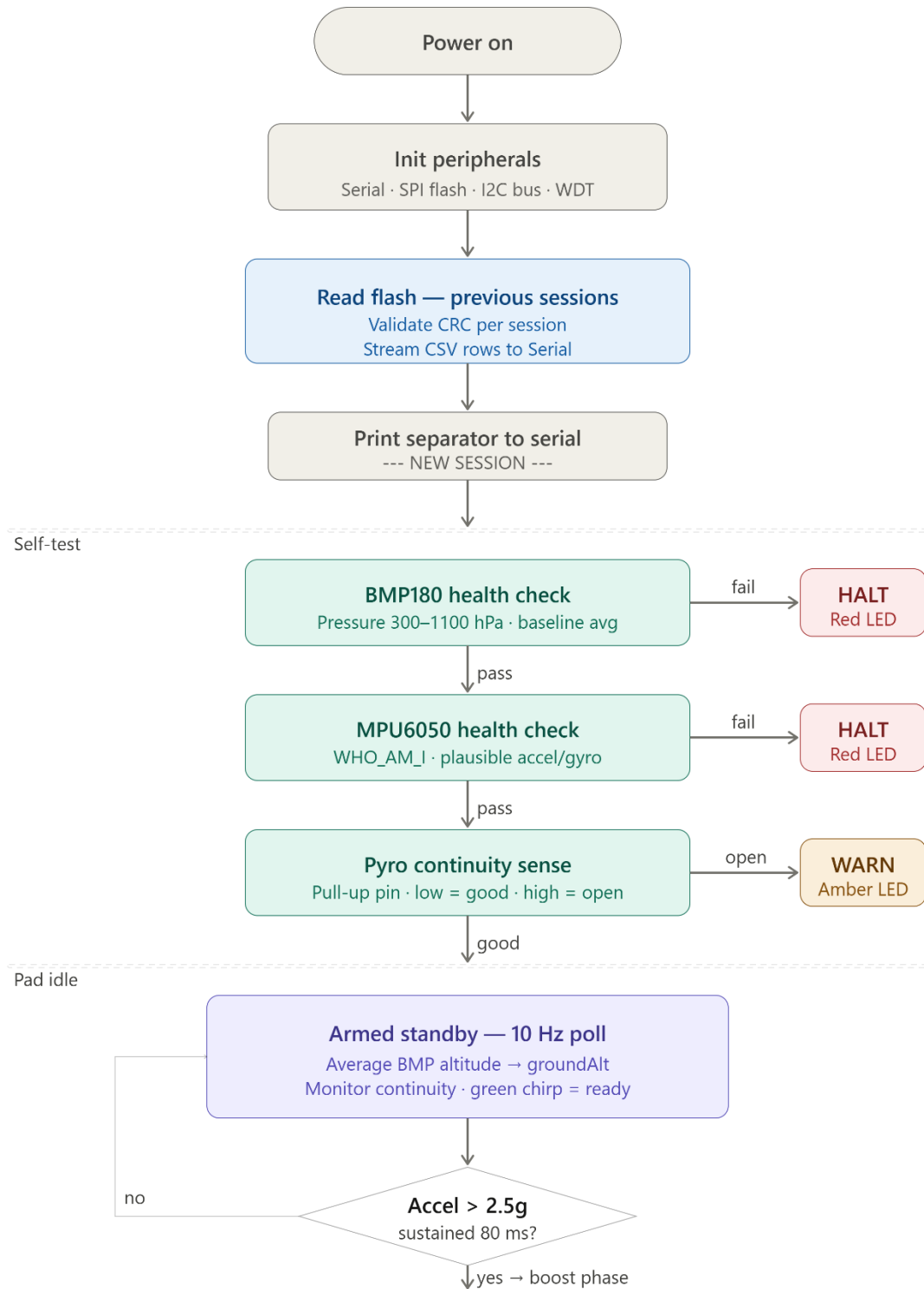


Diagram 2 — Boost and coast phase: from launch confirm through motor burnout, data logging, and the dual-sensor apogee watch loop.

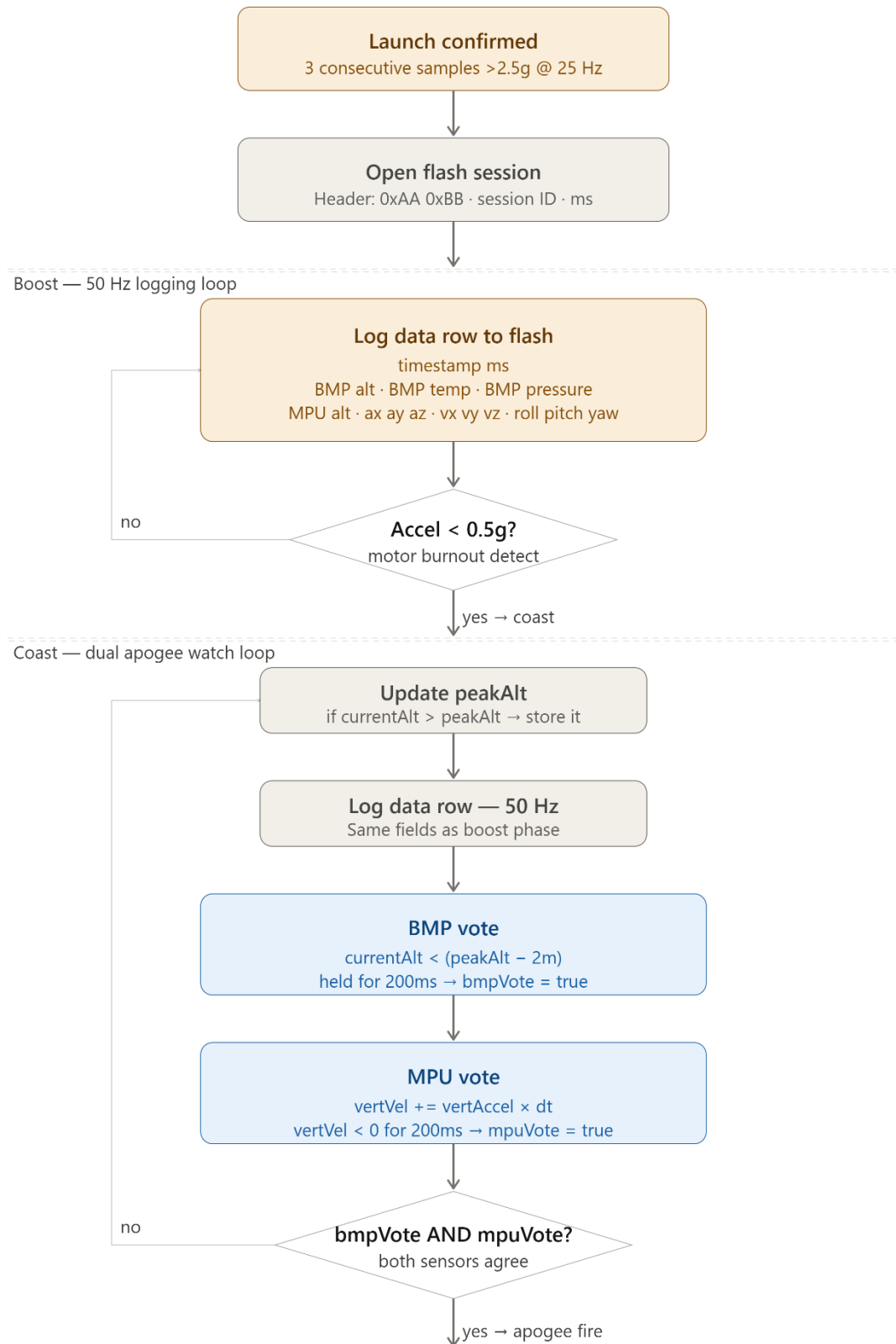
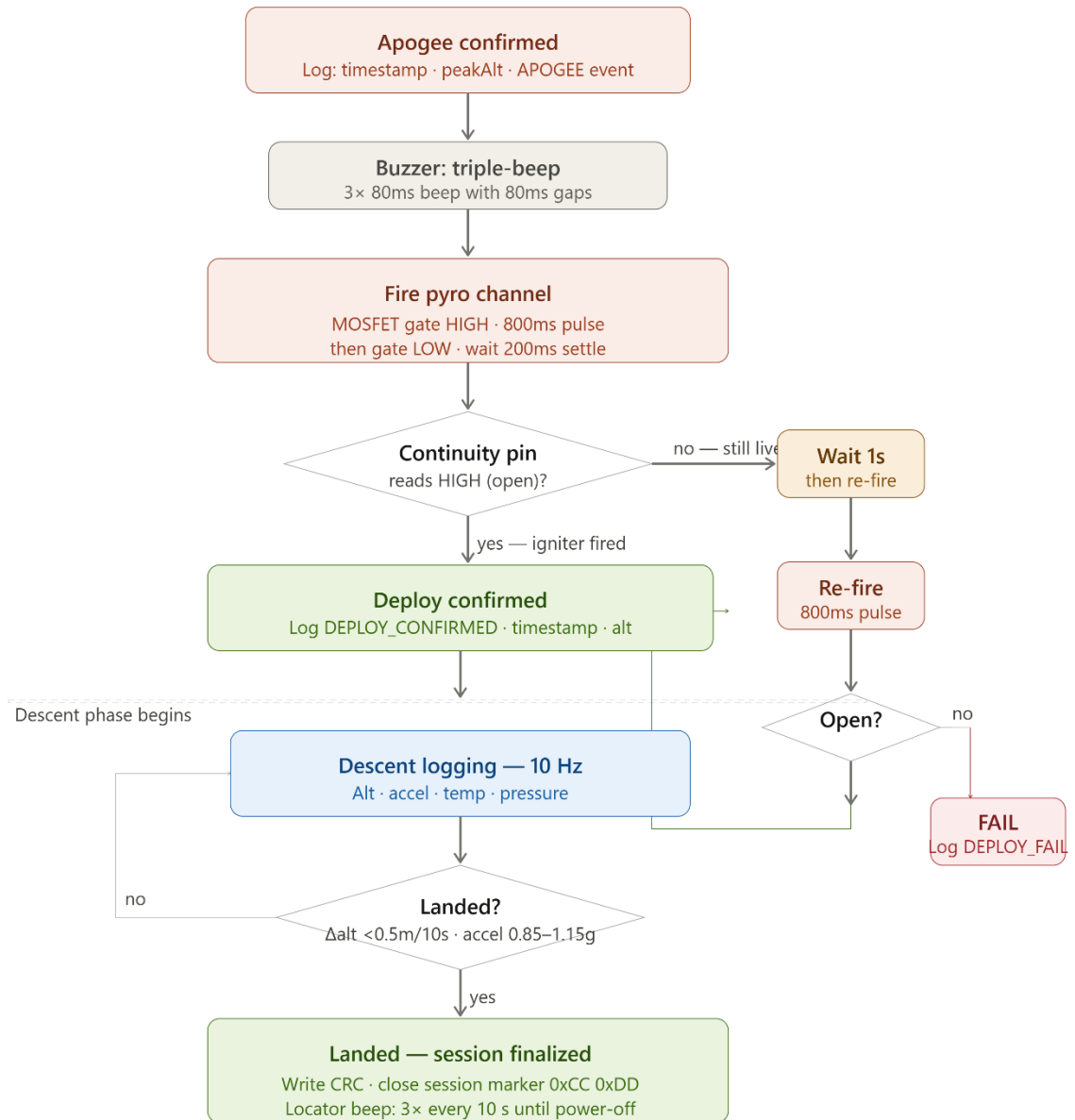


Diagram 3 — Apogee fire and pyro verification: the MOSFET pulse, continuity re-check, re-fire logic, and fail logging.



PTO

Diagram 4 — Watchdog and safety rails: the cross-cutting hardware guards that run in parallel across all phases.

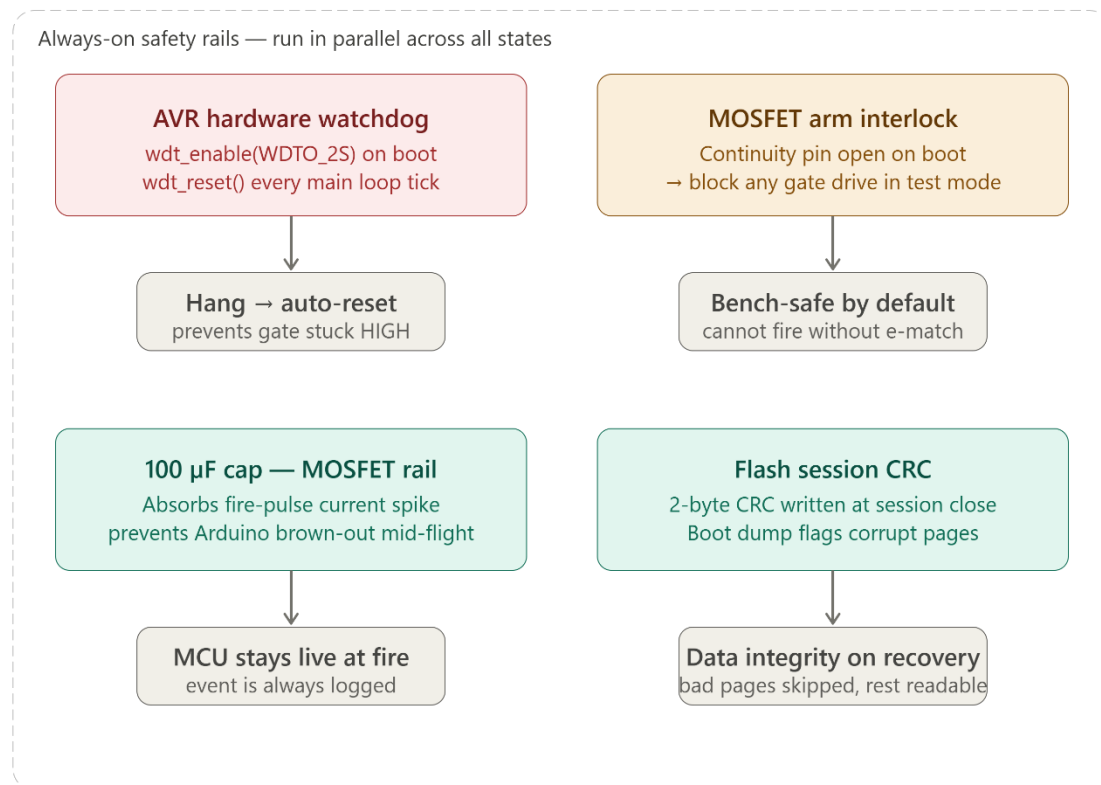


Diagram 1 covers power-on through armed standby — the FC readies sensors, dumps old data, gates through three self-test checks, then sits in the pad idle loop until the launch accelerometer threshold trips.

Diagram 2 is the flight core — the 50 Hz boost logging loop with the burnout detect gate, then the coast phase where the BMP and MPU independently accumulate their apogee votes. Neither sensor alone can trigger the fire.

Diagram 3 is the deployment sequence — the MOSFET pulse, the post-fire continuity re-check, the single re-fire attempt if the igniter didn't pop, and finally the descent and landing detection leading to session finalization and the locator beep.

Diagram 4 shows the four cross-cutting hardware guards that don't belong to any single phase — the AVR watchdog (prevents a firmware hang from locking the gate open), the bench interlock (bench-safe without an e-match connected), the 100µF capacitor (absorbs the pyro current spike so the Arduino doesn't brown-out mid-fire), and the CRC system (ensures corrupted flash pages are flagged rather than silently misread).